

Graph Neural Networks for Predicting Interatomic Potential Errors from Crystal Structure Topology: A Comprehensive Review

1. Introduction and Problem Context

1.1 The GLIM Platform and JARVIS-FF Dataset

The **Graph Learning for Interatomic Models (GLIM)** platform represents a transformative resource for computational materials science, aggregating comprehensive error data for **23 distinct interatomic potentials** evaluated across approximately **12,000 materials** from the **JARVIS-FF (Joint Automated Repository for Various Integrated Simulations—Force Field)** database ([PyPI](#)). This dataset enables systematic, quantitative analysis of how diverse empirical and machine learning potentials perform across chemical and structural spaces that span the periodic table, with particular emphasis on **elastic constant predictions** derived from high-throughput density functional theory (DFT) calculations.

The **JARVIS-FF framework**, developed at the National Institute of Standards and Technology (NIST), provides automated infrastructure for consistent property evaluation across force fields, enabling direct comparison of predictions against DFT reference values ([Github](#)). The underlying **JARVIS-DFT dataset** encompasses approximately **75,000 materials** with comprehensive property calculations, from which GLIM derives its error metrics by quantifying discrepancies between approximate potential predictions and these high-accuracy references ([RSC Publishing](#)). The integration with **JARVIS-Tools** facilitates seamless data preprocessing, crystal structure manipulation, and model deployment pipelines that are essential for production machine learning workflows.

The error data structure in GLIM is notably granular: rather than scalar error metrics, the platform maintains **per-potential error vectors** capturing deviations across multiple elastic constant components. For each material-potential pair, errors may be reported for **bulk modulus (K)**, **shear modulus (G)**, **Young's modulus (E)**, and **individual elastic tensor components (C_{ij})**, creating a rich, multi-dimensional supervision signal. This vector-valued representation is essential because different interatomic potentials exhibit **characteristic error patterns**—systematic overestimation of bulk moduli for ionic compounds, failure to capture shear anisotropy in layered materials, or catastrophic errors near phase boundaries—that a unified model can learn to recognize and predict from crystal structure topology alone.

The 23 potentials span four major families with fundamentally different mathematical structures: **empirical potentials** (EAM, MEAM variants) based on electron density embeddings; **reactive potentials** (ReaxFF, bond-order formulations) enabling dynamic bond breaking; **machine learning potentials** (SNAP, ACE, neural network potentials) with data-driven functional forms; and **ab initio-derived potentials** (tight-binding, DFT-parameterized models) incorporating simplified quantum mechanical treatments ([OpenReview](#)) ([Nature](#)). This diversity creates both challenges for model generalization and opportunities for transfer learning across related formalisms.

1.2 Primary Objectives for GNN Development

The development of graph neural networks for the GLIM platform pursues three interconnected objectives that collectively address critical needs in materials simulation and force field selection.

High prediction accuracy for error vector estimation enables reliable pre-screening of potential performance before expensive molecular dynamics or Monte Carlo simulations are undertaken. The economic and computational benefits are substantial: accurate error prediction can prevent wasted resources

on simulations where the chosen potential is likely to produce unreliable results. For elastic constants, which require careful finite-difference or stress-strain calculations, the cost of a single evaluation can range from minutes to hours depending on system size, making predictive screening invaluable. The target accuracy should approach or exceed the **10-15% mean relative error** achieved by state-of-the-art GNNs for direct elastic property prediction, translated to the error-on-error prediction task ([Springer](#)) .

Interpretability: identifying structural motifs causing potential failures transforms the GNN from a black-box predictor into a diagnostic tool that can guide force field development and selection. By identifying which atomic environments, bond configurations, or local symmetry elements contribute most strongly to prediction errors, researchers can: (1) select alternative potentials better suited to specific materials classes; (2) flag regions of chemical space where existing potentials are unreliable; and (3) prioritize improvements to potential functional forms or parameterizations ([arXiv.org](#)) . Attention mechanisms and gradient-based attribution methods provide the technical foundation for this interpretability, enabling visualization of structure-error relationships that can be validated against known physical limitations of interatomic potentials.

Transferability: generalization across diverse force field families ensures that models trained on the available 23 potentials can accommodate newly developed force fields without extensive retraining. This capability is essential in a rapidly evolving field where new machine learning potentials are introduced monthly and empirical potentials are continuously refined. The multi-task learning framework, where shared representations are learned across all potentials with task-specific output heads, provides a principled approach to achieving this transferability. The ultimate goal is **few-shot adaptation**: given error data for a new potential on only tens or hundreds of materials, the model should achieve useful prediction accuracy by leveraging structural knowledge transferred from related potentials ([Github](#)) .

2. Crystal Graph Representations and Foundational Architectures

2.1 Graph Construction Methodologies for Crystals

The transformation of crystalline materials into graph representations constitutes the foundational pre-processing step that constrains all subsequent learning. The specific choices made in this transformation—how atoms are featurized, which interactions are represented as edges, and how periodicity is handled—fundamentally determine what structural information is available to the neural network and what is irretrievably discarded.

Node features: atomic number encoding, one-hot representations, and learned embeddings.

The standard approach, established by CGCNN, employs a **92-dimensional feature vector** for each element comprising: group number, period, electronegativity, covalent radius, valence electrons, ionization energy, electron affinity, block (s/p/d/f), and other atomic properties derived from established chemical databases ([PyG Documentation](#)) . These features are initialized from physical data and can be either frozen or fine-tuned during training. Subsequent work has demonstrated that **learned embeddings**—initialized randomly or from pre-trained representations and optimized end-to-end—often outperform static features for downstream prediction tasks, as they can adapt to the specific structural and chemical patterns relevant to the target property. The **S-CGCNN modification** reduced the feature dimension to 18 through an “adaptable encoder” approach, achieving comparable accuracy with less than 5% of the parameters of deeper architectures ([Wiley Online Library](#)) . For the GLIM application, **64-128 dimensional learned embeddings** provide a reasonable balance between expressiveness and computational efficiency.

Edge definition strategies: distance cutoffs, k-nearest neighbors, and Voronoi tessellation.

Three primary approaches define connectivity in crystal graphs, each with distinct advantages:

Method	Description	Advantages	Disadvantages	Typical Parameters
Distance cutoff	Connect atoms within radius r_{cut}	Physically motivated, captures all interactions in range	Variable neighbor count, disconnected graphs possible	$r_{\text{cut}} = 5\text{-}8 \text{ \AA}$
k-nearest neighbors	Connect to k closest atoms	Fixed neighbor count, predictable memory	Variable edge distances, density-dependent	$k = 8\text{-}16$
Voronoi tessellation	Connect atoms sharing cell faces	Parameter-free, geometrically natural	Unbounded edges, computational cost	—

The **ALIGNN architecture** demonstrates that including **up to 12 nearest neighbors** with careful periodic boundary treatment yields optimal performance for elastic property prediction, with the line graph construction additionally capturing bond-angle information critical for directional properties ([Springer](#)) . The **Distribution Graph (DG)** modification reduces graph size by **28%** while maintaining accuracy through intelligent selection of symmetry-equivalent representations ([Springer](#)) .

Periodic boundary condition handling requires explicit treatment of crystal periodicity. Standard practice constructs supercells sufficient to encompass all edges within the chosen cutoff, with edges created to periodic images as needed. The minimum image convention ensures that for each atom pair, the closest periodic image is selected. More sophisticated approaches encode periodicity directly into message passing operations, ensuring that predictions are strictly invariant to unit cell choice and supercell size.

Gaussian expansion of interatomic distances transforms continuous spatial relationships into finite-dimensional edge features suitable for neural network processing. Following SchNet, **40 Gaussian basis functions** centered at uniformly spaced distances with learned width parameters enable smooth interpolation between discrete training examples ([Github](#)) . The radial basis function expansion in MatTen employs **spherical Bessel functions** that provide better asymptotic properties for long-range interactions ([arXiv.org](#)) . This continuous representation is essential for properties like elastic constants that depend sensitively on small structural perturbations.

2.2 CGCNN: Crystal Graph Convolutional Neural Network

The **Crystal Graph Convolutional Neural Network (CGCNN)**, introduced by **Xie and Grossman in 2018**, established the foundational architecture for deep learning on crystal structures and remains widely adopted due to its conceptual simplicity and strong empirical performance ([PyG Documentation](#)) . CGCNN's design philosophy centers on **direct learning from crystal topology without manual feature engineering**, using connectivity and distance information as the primary input representation.

The architecture comprises three main components: **graph construction**, **convolutional feature extraction**, and **property prediction**. Atoms are represented as nodes with 92-dimensional elemental feature vectors. Edges connect all atom pairs within a specified cutoff radius (typically **8 Å**), with edge features encoding Gaussian-expanded distances. The convolutional operation updates node features through **element-wise multiplication of transformed neighbor features and edge-gated activations**:

$$\mathbf{x}'_i = \mathbf{x}_i + \sum_{j \in \mathcal{N}(i)} \sigma(\mathbf{z}_{i,j} \mathbf{W}_f + \mathbf{b}_f) \odot g(\mathbf{z}_{i,j} \mathbf{W}_s + \mathbf{b}_s)$$

where $\mathbf{z}_{i,j} = [\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{i,j}]$ concatenates central node, neighbor node, and edge features; σ is the sigmoid gating function; and g is the softplus activation ([PyG Documentation](#)). This formulation enables information propagation across the crystal graph while maintaining differentiability with respect to both node and edge features.

After **3-5 convolutional layers**, node features are pooled through **simple mean aggregation** to produce a graph-level representation, which is processed by fully-connected layers for property prediction. The mean pooling enforces **intensive property scaling**—predictions are independent of system size—appropriate for elastic moduli.

Limitations for anisotropic properties stem from CGCNN’s reliance on **scalar distance features without explicit angular information**. Structures with identical radial distribution functions but different three-dimensional arrangements—such as diamond cubic versus hexagonal close packing—are indistinguishable to CGCNN, limiting accuracy for shear moduli and full elasticity tensors where directional bonding is critical. Additionally, the **rotational invariance** of distance-only features, while physically correct for scalar properties, prevents direct prediction of tensor quantities that must transform covariantly under coordinate rotations. These limitations motivated subsequent architectures incorporating explicit geometric information.

2.3 MEGNet: MatErials Graph Network

MEGNet, developed by [Chen et al.](#), introduced a **modular architecture** that explicitly separates processing pathways for **atoms, bonds, and global state variables**, enabling more flexible information flow and improved interpretability ([PyPI](#)). This tripartite structure reflects the physical intuition that material properties emerge from interactions across multiple scales—from individual atoms to local bonding environments to collective system characteristics.

The **global state network** is MEGNet’s distinctive innovation, introducing a set of features representing whole-crystal properties that can include composition statistics, lattice parameters, temperature, pressure, or—critically for the GLIM application—the **specific interatomic potential being evaluated**. The global features are broadcast to all nodes and edges, modulating their transformations and enabling the model to adapt its predictions based on potential-specific information. This mechanism provides a natural pathway for **multi-fidelity learning**, where data from sources with varying accuracy can be jointly leveraged by encoding data source identity in the global state ([PyPI](#)).

Set2Set pooling addresses the challenge of producing fixed-size outputs from variable-size graphs through an **order-invariant LSTM-based aggregation**. The processing chain generates query vectors that attend to node features over multiple iterations:

$$\mathbf{q}_t = \text{LSTM}(\mathbf{c}_{t-1}), \quad \alpha_{i,t} = \text{softmax}(\mathbf{q}_t^T \mathbf{x}_i), \quad \mathbf{r}_t = \sum_i \alpha_{i,t} \mathbf{x}_i$$

with **3-6 processing steps** typically optimal for materials applications. This iterative attention enables progressive refinement of the graph representation, focusing on increasingly relevant structural features. The attention weights provide **interpretability**: which atoms most influence the final prediction can be extracted and visualized.

MEGNet demonstrated **state-of-the-art performance** on formation energy, band gap, and elastic property prediction when introduced, with subsequent extensions including **M3GNet** (incorporating three-body interactions) achieving screening of **31 million hypothetical crystal structures** to identify **1.8 million potentially stable new materials (PyPI)**. For the GLIM application, MEGNet’s modular design and global state conditioning provide a strong foundation for multi-potential error prediction.

2.4 SchNet: Continuous-Filter Convolutional Neural Network

SchNet introduced **continuous-filter convolutions** that operate directly on continuous spatial coordinates rather than precomputed graph structures, enabling **end-to-end differentiability** with respect to atomic positions ([Github](#)). This design is particularly advantageous for force prediction and properties requiring spatial gradients, where automatic differentiation through the network provides efficient, consistent derivative calculations.

The **filter-generating network** produces continuous convolution kernels as functions of interatomic distance. These filters are constructed from **radial basis functions** multiplied by learnable coefficients, creating smooth, differentiable functions that can interpolate between training examples. The convolution operation integrates these filters over neighboring atoms:

$$\mathbf{x}_i^{(l+1)} = \sum_{j \in \mathcal{N}(i)} \mathbf{x}_j^{(l)} \odot \mathbf{W}^{(l)}(r_{ij})$$

where $\mathbf{W}^{(l)}(r_{ij})$ is the continuous filter evaluated at distance r_{ij} . This formulation maintains **translational invariance** and **approximate rotational invariance** through distance-based features, with extensions to full **E(3)-equivariance** through vector-valued features.

For **elastic constant prediction**, SchNet’s explicit treatment of spatial derivatives enables efficient computation of **stress tensors** and **elastic coefficients** through nested automatic differentiation. The smooth, differentiable potential energy surfaces learned by SchNet are particularly well-suited to the small-strain regime where elastic response is defined. However, like CGCNN, the original SchNet formulation predicts **scalar energies and forces** rather than full tensorial properties, requiring architectural extensions for complete elasticity tensor prediction.

2.5 DimeNet: Directional Message Passing Network

DimeNet addressed the critical limitation of **distance-only architectures** by incorporating **explicit directional information through angle embeddings** ([Github](#)). The key insight is that many material properties, particularly mechanical anisotropy, depend sensitively on **three-dimensional atomic arrangement** rather than merely pairwise distances.

The architecture implements **directional message passing over directed edges**, where messages between atoms are conditioned on both the direct interaction and the **angular environment** formed by neighboring atoms. For a message from atom j to atom i , DimeNet incorporates angles θ_{ijk} formed with other neighbors k , enabling distinction of tetrahedral, octahedral, and other coordination geometries that are indistinguishable in radial distribution functions.

Bessel basis functions provide efficient, orthogonal expansions for both radial and angular features. **Spherical Bessel functions** with learned cutoff parameters capture distance-dependent interactions, while **spherical harmonics** encode directional dependence with guaranteed rotational properties. The combined representation enables DimeNet to distinguish structures with identical bond length distributions but different angular configurations—a common occurrence in polymorphic materials systems.

The computational cost of DimeNet’s directional message passing is **higher than radial-only alternatives**, scaling as $O(N \cdot k^2)$ compared to $O(N \cdot k)$ for SchNet or CGCNN, due to the explicit treatment of angle triplets. This cost is justified for properties where angular dependencies are critical, including **shear moduli** and **full elasticity tensors**. The **DimeNet++ variant** reduces computational overhead through optimized message passing schedules while maintaining accuracy ([Github](#)) .

2.6 GemNet: Geometric Message Passing Network

GemNet extends directional message passing to **higher-order geometric features**, incorporating **four-body dihedral angles** for comprehensive characterization of local atomic environments ([Github](#)) . This extension enables unambiguous recognition of complex three-dimensional arrangements that may be indistinguishable to models using only distances and bond angles.

The key technical innovation is **efficient factorization of higher-order interactions**. Rather than enumerating all multi-body terms combinatorially—which would lead to exponential complexity—GemNet recursively constructs higher-order features from lower-order ones through learned combinations. This factorization maintains expressiveness while controlling computational cost, enabling application to practical materials systems.

For **large-scale materials screening**, GemNet’s computational requirements must be carefully balanced against accuracy gains. Empirical studies suggest that for **bulk modulus prediction**, simpler architectures may provide comparable accuracy at lower cost, while for **shear modulus and full elastic tensor prediction**, GemNet’s geometric richness yields **substantial improvements**. Optimized kernel implementations and batch processing enable throughput competitive with simpler models when amortized over large datasets.

2.7 Equivariant GNNs for Tensor Properties

The prediction of **elasticity tensors**—fourth-rank tensors with complex symmetry constraints—demands architectures that respect the **SO(3) rotational symmetry** of physical space. **Equivariant graph neural networks**, exemplified by the **MatTen (Materials Tensor) architecture**, achieve this by constructing features that transform according to **irreducible representations of the rotation group** ([arXiv.org](#)) .

The theoretical foundation lies in **group representation theory**: arbitrary tensors decompose into irreps labeled by angular momentum quantum number l —scalars ($l = 0$), vectors ($l = 1$), and higher-rank tensors ($l \geq 2$). Equivariant neural networks maintain this decomposition throughout, with **Clebsch-Gordan coefficients** governing how irreps combine in message passing operations. For elastic constants, the relevant irreps span $l = 0, 2, 4$, requiring the network to produce and combine features at these angular momenta.

MatTen’s architecture comprises four modular components: **position embedding** (radial and spherical expansions of displacement vectors), **atom embedding** (irreducible feature initialization), **interaction blocks** (equivariant tensor product convolutions), and **output head** (elasticity tensor assembly) ([arXiv.org](#)) . The **SO(3)-equivariance guarantee** ensures that predicted tensors transform correctly under arbitrary rotations, eliminating spurious anisotropy from broken symmetry.

Empirical performance on the Materials Project dataset demonstrates MatTen’s advantages: **MAE of 7.37 GPa for bulk modulus, 8.38 GPa for shear modulus, and 20.59 GPa for Young’s modulus**, with **30-50% error reductions for anisotropic components** compared to non-equivariant alternatives ([arXiv.org](#)) . Critically, MatTen enables **screening for extreme elastic properties**—

identifying 100 materials with directional Young’s modulus exceeding 600 GPa, including **11 unconventional polymorphs of cubic metals** where maximum stiffness occurs in unexpected crystallographic directions. For the GLIM application, equivariant architectures provide the most physically principled approach to **tensor-valued error prediction**, though at increased computational cost.

3. Multi-Task Learning for Error Vector Prediction

3.1 Problem Formulation

The GLIM platform’s objective—predicting **23-dimensional error vectors** for elastic constants—represents a fundamental departure from conventional single-property prediction. This multi-output structure enables **joint learning across interatomic potentials**, exploiting correlations between error patterns to improve data efficiency and generalization.

The mathematical formulation treats each material-potential pair as a training example with **shared input** (crystal graph) but **distinct outputs** (error vectors). Given dataset $\mathcal{D} = \{(G_i, \mathbf{y}_i)\}$ where G_i is the crystal graph and $\mathbf{y}_i \in \mathbb{R}^{23}$ is the vector of errors across all potentials, the learning objective is to find $f : \mathcal{G} \rightarrow \mathbb{R}^{23}$ minimizing expected prediction error. The key insight is that **error correlations reflect shared structural determinants**: coordination environments that cause EAM failures often cause related failures in MEAM potentials; charge transfer regions problematic for empirical potentials are similarly challenging for reactive force fields.

Shared representation learning amortizes the cost of structure understanding across all 23 prediction tasks. A common encoder learns to extract features—coordination polyhedra, bond length distributions, chemical ordering patterns—that predict potential accuracy regardless of specific functional form. Task-specific heads then map these shared features to individual potential errors, learning idiosyncratic variations. This architecture enables **transfer from data-rich to data-scarce potentials**: a potential with limited evaluation history can achieve reasonable accuracy by leveraging structural knowledge transferred from extensively evaluated related potentials.

Task relationships exhibit hierarchical structure: at the coarsest level, potential families (EAM, ReaxFF, ML potentials, ab initio-derived) share common functional forms; within families, specific parameterizations create finer-grained correlations. This structure motivates **hierarchical multi-task architectures** with family-specific intermediate representations, though empirical validation is required to determine whether hand-designed or learned hierarchies perform better.

3.2 Architecture Design Strategies

The **shared encoder with task-specific output heads** represents the most widely adopted and empirically effective architecture. The encoder—comprising **3-6 graph convolutional layers**—processes the crystal graph to produce node and graph-level representations. Each of **23 parallel output heads**, typically **2-3 layer MLPs**, transforms these shared features into potential-specific error predictions. Head complexity can vary with prior knowledge: simple linear heads for well-understood empirical potentials, deeper networks for complex machine learning potentials with intricate error surfaces.

Sharing Strategy	Architecture	Parameters	Best For
Hard sharing	Single encoder, separate heads	Minimal	Highly related tasks, limited data per task

Table 2 – continued

Sharing Strategy	Architecture	Parameters	Best For
Soft sharing	Separate encoders with similarity regularization	Moderate	Moderately related tasks with distinct patterns
Hierarchical	Shared low-level, family-specific intermediate, task-specific heads	Flexible	Clear task groupings with known structure

Hard parameter sharing maximizes efficiency and enforces strong regularization: the encoder must satisfy all 23 objectives simultaneously, preventing overfitting to any single potential. **Soft parameter sharing** through cross-stitch networks or sluice networks provides flexibility when potentials have divergent requirements, at increased parameter count. For GLIM, **hard sharing with task-specific batch normalization** has proven effective, enabling adaptation to different error scales while maintaining representation sharing ([Github](#)) .

Hierarchical multi-task structures explicitly model potential family relationships. The shared encoder feeds into **family-specific modules** (EAM processor, ReaxFF processor, ML potential processor), which in turn connect to **individual potential heads**. This architecture matches expected correlation structure and can improve sample efficiency when family assignments are accurate. However, rigid hierarchies may limit adaptation to unexpected cross-family relationships, suggesting that **learned routing mechanisms** or **attention-based task grouping** may ultimately outperform fixed structures.

3.3 Loss Function Design

The **weighted sum of per-potential losses** provides the simplest multi-task objective:

$$\mathcal{L} = \sum_{p=1}^{23} w_p \cdot \mathcal{L}_p(f_p(\mathbf{h}), y_p)$$

with weights w_p balancing task contributions. **Uniform weighting** often fails when error scales differ dramatically across potentials. **Inverse variance weighting** with $w_p \propto 1/\sigma_p^2$ normalizes tasks to comparable scales. **Learned weighting** through task uncertainty provides adaptive balancing without manual tuning.

Uncertainty-weighted task losses, following Kendall et al., learn per-task uncertainties σ_p that automatically adapt weights during training ([ACS Publications](#)) :

$$\mathcal{L} = \sum_{p=1}^{23} \left[\frac{1}{2\sigma_p^2} \mathcal{L}_p + \log \sigma_p \right]$$

The first term encourages accurate predictions with appropriate confidence; the second term prevents collapse to zero uncertainty. Tasks with high inherent noise automatically receive reduced weight, while well-predictable tasks are emphasized. For GLIM, where some potentials exhibit **variable convergence behavior** or **numerical instabilities**, uncertainty weighting provides robustness to label quality variations.

Correlation-aware loss terms explicitly model dependencies between potential errors. Rather than treating the 23 errors as independent, a **multivariate Gaussian likelihood** with learned covariance captures systematic correlations: errors for related potentials should covary positively, while complementary potentials may show negative correlations. Low-rank covariance approximations control computational cost. Preliminary experiments suggest **10-15% accuracy improvements** for data-scarce potentials when correlation structure is modeled.

3.4 Implementation Considerations

Batch construction with multiple potential labels per material requires handling **missing data patterns**: not all 23 potentials have been evaluated for all 12,000 materials. Standard practice employs **masked loss computation** where only observed error values contribute to gradients. The PyTorch Geometric Data object is extended with multi-dimensional target tensors and corresponding mask indicators, enabling efficient vectorized operations despite variable observation patterns.

Handling missing error data extends to evaluation protocols: cross-validation must respect missingness patterns to avoid optimistic bias. **Stratified sampling** ensures representative coverage of potential-material combinations in all folds. For materials with sparse potential coverage, **mean absolute error computed only on observed values** provides more reliable estimates than imputation-based approaches.

Gradient balancing techniques address training stability when 23 task gradients compete for shared parameters. **GradNorm** dynamically adjusts task weights to balance gradient magnitudes; **PCGrad** projects conflicting gradients onto orthogonal directions to reduce interference. For GLIM, **gradient clipping with task-specific thresholds**—set based on initial gradient statistics—has proven essential for stable convergence.

4. Uncertainty-Aware GNN Architectures

4.1 Sources of Uncertainty in Error Prediction

Reliable uncertainty quantification is essential for practical deployment of GNN-based error prediction, enabling **risk-aware decision-making**, **active learning**, and **model diagnostics** ([arXiv.org](#)) ([ACS Publications](#)) . Three distinct uncertainty sources must be distinguished:

Uncertainty Type	Source	Reducible?	Modeling Approach
Aleatoric	Inherent noise in error measurement (DFT limitations, potential instabilities, finite-size effects)	No	Heteroscedastic output layers with learned variance

Table 3 – continued

Uncertainty Type	Source	Reducible?	Modeling Approach
Epistemic	Model knowledge gaps from limited training data or capacity	Yes (with more data/better models)	MC dropout, deep ensembles, Bayesian NNs
Distributional	Out-of-distribution inputs (novel structures, compositions, potentials)	Partially (with better coverage)	Density estimation, anomaly detection, OOD detectors

Aleatoric uncertainty captures irreducible noise: even perfect knowledge of crystal structure cannot eliminate fundamental randomness in error generation. DFT reference calculations have **exchange-correlation functional errors**, **basis set incompleteness**, and **convergence approximations**; force field evaluations may suffer from **numerical instabilities** or **finite-size artifacts**. This uncertainty varies with material characteristics—complex oxides with variable oxidation states exhibit higher inherent unpredictability than simple elemental metals.

Epistemic uncertainty reflects model ignorance: regions of chemical space with limited training coverage, novel crystal structures, or newly introduced potentials with sparse evaluation history. Unlike aleatoric uncertainty, epistemic uncertainty **decreases with additional data**, making it a valuable guide for **active learning**—prioritizing which materials and potentials to evaluate next.

Distributional uncertainty addresses the critical question of whether test inputs belong to the training distribution at all. Materials with **unprecedented compositions**, **exotic bonding motifs**, or **extreme conditions** may elicit confident but arbitrary predictions from models operating outside their valid domain. Reliable OOD detection enables appropriate fallback strategies: deferring to expensive but reliable calculations, flagging predictions for human review, or triggering targeted data collection.

4.2 Aleatoric Uncertainty Quantification

Heteroscedastic output layers extend standard regression to predict both mean and variance for each error component. For potential p , the model outputs $(\hat{\mu}_p, \hat{\sigma}_p^2)$ with **negative log-likelihood loss**:

$$\mathcal{L}_p = \frac{1}{2} \left[\log \hat{\sigma}_p^2 + \frac{(y_p - \hat{\mu}_p)^2}{\hat{\sigma}_p^2} \right]$$

This formulation encourages accurate mean predictions while penalizing overconfident uncertainty estimates that fail to cover observed errors. The learned variance function $\hat{\sigma}_p^2(x)$ captures **input-dependent noise**: materials with complex phase boundaries or significant anharmonicity receive appropriately elevated uncertainty.

Laplace versus Gaussian likelihoods reflect different assumptions about error distributions. **Gaussian** (squared error) is sensitive to outliers and appropriate for symmetric, unimodal errors. **Laplace** (absolute error) with heavier tails is more **robust to occasional large deviations** from convergence

failures or numerical instabilities. **Student-t** distributions provide additional flexibility with learned degrees of freedom. Empirical analysis of GLIM residual distributions should guide selection; preliminary evidence suggests **Laplace or Student-t** may better capture the occasional catastrophic failures observed for certain potentials.

Data-dependent uncertainty modeling enables the GNN to learn which structural features predict unreliable error measurements. For example, the model may learn that **EAM uncertainty increases with charge transfer magnitude**, while **ReaxFF uncertainty correlates with bond order variation**. These learned structure-uncertainty relationships provide actionable guidance: when high uncertainty is predicted, users can understand which features drive this assessment and select alternative approaches accordingly.

4.3 Epistemic Uncertainty Estimation

Monte Carlo dropout provides computationally efficient approximate Bayesian inference ([arXiv.org](https://arxiv.org)). By retaining dropout at test time with the same rate used during training, **multiple stochastic forward passes** generate prediction distributions whose variance estimates epistemic uncertainty. For GLIM, **50-100 forward passes** provide stable estimates with modest overhead. Dropout placement requires care: **node feature dropout between convolutional layers** captures representation uncertainty; **output head dropout** captures task-specific uncertainty.

Deep ensemble methods train **5-10 independent models** with different initializations and aggregate predictions. Ensemble variance provides well-calibrated epistemic uncertainty, with **variance reduction from averaging** typically improving point predictions over single models. The primary cost is training time, mitigated by parallel training. For production GLIM deployment, ensembles provide the most reliable uncertainty estimates and are recommended for high-stakes decisions.

Method	Computational Cost	Calibration Quality	Implementation Complexity
MC Dropout	Low (10-100× inference)	Moderate	Simple: single model, dropout at test time
Deep Ensembles	High (5-10× training)	Best	Moderate: multiple independent trainings
Variational BNNs	Very high	Good	Complex: approximate posterior inference
Evidential Networks	Lowest (single forward pass)	Moderate	Moderate: specialized output layers

Bayesian neural networks with variational inference place distributions over weights and approximate posteriors through optimization. While theoretically principled, computational challenges with millions of GNN parameters have limited practical adoption. Recent advances in **stochastic gradient Hamiltonian Monte Carlo** may improve scalability but remain research-level for materials applications.

Evidential deep learning with Dirichlet-based uncertainty predicts distribution parameters directly, enabling **single-pass uncertainty estimation** comparable in cost to deterministic prediction ([ACS Publications](#)). For regression, **Normal-Inverse-Gamma** formulations provide closed-form predictive distributions. The computational efficiency is attractive for large-scale screening, though calibration may require more careful tuning than ensemble methods.

4.4 Combined Uncertainty Frameworks

Joint modeling of aleatoric and epistemic components provides complete uncertainty characterization. The **total predictive variance decomposes** as:

$$\text{Var}(y|x) = \underbrace{\mathbb{E}_\theta[\sigma^2(x;\theta)]}_{\text{aleatoric}} + \underbrace{\text{Var}_\theta[\mu(x;\theta)]}_{\text{epistemic}}$$

With MC dropout or ensembles, this becomes:

$$\hat{\sigma}^2_{\text{total}} = \frac{1}{T} \sum_{t=1}^T \hat{\sigma}^2(x; \theta_t) + \frac{1}{T} \sum_{t=1}^T (\hat{\mu}(x; \theta_t) - \bar{\mu})^2$$

This decomposition enables **targeted interventions**: high aleatoric uncertainty suggests fundamental unpredictability requiring alternative potentials or improved references; high epistemic uncertainty indicates where **additional data collection** would be most valuable.

Calibration techniques ensure predicted uncertainties match observed error frequencies. **Temperature scaling**—learning a single scalar to adjust predicted variances—provides simple, effective calibration. **Isotonic regression** or **Platt scaling** learn non-linear mappings for more complex miscalibration patterns. For GLIM, calibration should be evaluated **per potential and per error component**, as patterns vary across tasks.

Uncertainty decomposition for actionable diagnostics extends to **feature attribution**: which structural aspects drive uncertainty? Methods like **SHAP for uncertainty** or **gradient-based uncertainty attribution** can identify that, for example, **undercoordination drives epistemic uncertainty for EAM potentials**, guiding targeted data collection or feature engineering.

5. Transfer Learning Across Force Field Families

5.1 Force Field Family Characterization

The **23 interatomic potentials** in GLIM span four major families with distinct theoretical foundations, functional forms, and characteristic failure modes:

Family	Representative Potentials	Core Formalism	Typical Strengths	Characteristic Limitations
Empirical	EAM, MEAM variants	Electron density embedding	Metals, alloys; computational efficiency	Charge transfer, directional bonding, surfaces
Reactive	ReaxFF, bond-order methods	Dynamic bond-order with charge equilibration	Chemical reactions, bond breaking/formation	Many parameters; accuracy outside training
Machine Learning	SNAP, ACE, NN potentials	Data-driven regression on descriptors	Near-DFT accuracy for similar structures	Extrapolation failures; training data requirements

Ab initio-derived	Tight-binding, DFT-fit	Simplified quantum mechanics	Intermediate accuracy and cost	Systematic errors from underlying approximations
--------------------------	------------------------	------------------------------	--------------------------------	--

Empirical potentials (EAM, MEAM) derive from density functional theory approximations with parameters fit to experimental or computational data. They excel for **metallic systems** with nearly free electron behavior but struggle with **charge transfer**, **directional covalent bonding**, and **surface energies** where the embedding function description breaks down. Error patterns are often **systematic and predictable** from coordination environment and electron density.

Reactive potentials (ReaxFF) enable simulation of **chemical reactions** through dynamic bond-order formalisms with **charge equilibration**. Their **hundreds of parameters per element** create flexibility but also **overfitting risks** and **unpredictable extrapolation**. Errors correlate with **bond order variation** and **charge transfer magnitude**, distinguishing them from empirical potentials.

Machine learning potentials (SNAP, ACE, neural networks) learn **directly from reference data** without prescribed functional forms. They achieve **near-DFT accuracy** for training-similar structures but exhibit **catastrophic failures when extrapolating** to unfamiliar chemical environments. Error patterns are **less systematic** and may depend sensitively on training data coverage.

Ab initio-derived potentials occupy an intermediate position, with **functional forms motivated by electronic structure theory** but simplified for efficiency. Their errors often **correlate with full DFT errors**, making them valuable for **relative rankings** where absolute accuracy is less critical.

5.2 Representation Strategies for Potential Type

Potential family as global graph-level feature provides the simplest integration. A **learned embedding vector** representing potential type is concatenated with pooled graph features before output layers, enabling family-specific predictions from shared structural representations. This requires minimal architectural modification and provides **interpretable potential relationships** through embedding space analysis.

Edge-type encoding via R-GCN (Relational Graph Convolutional Networks) treats different potentials as different edge types in a multi-relational graph (PeerJ). The message passing rule becomes:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{c_{i,r}} \mathbf{W}_r^{(l)} \mathbf{h}_j^{(l)} + \mathbf{W}_0^{(l)} \mathbf{h}_i^{(l)} \right)$$

where $\mathcal{R}$ is the set of relations (potential types) and $\mathbf{W}_r$ are relation-specific weights. **Basis decomposition** $\mathbf{W}_r = \sum_{b=1}^B a_{rb} \mathbf{V}_b$ reduces parameters when the number of potentials is large. For GLIM with 23 potentials, **4-8 bases** typically suffice.

Potential-specific embedding layers with shared structural encoders strike a balance: a **common GNN encoder** processes crystal graphs; **potential-specific MLPs** transform embeddings before prediction. This enables efficient structure sharing while maintaining adaptation flexibility:

$$\mathbf{h}_i^{(p)} = \text{MLP}_p(\mathbf{h}_i^{(L)}), \quad \hat{y}_p = f_p(\text{Pool}(\{\mathbf{h}_i^{(p)}\}))$$

Meta-learning approaches such as **MAML (Model-Agnostic Meta-Learning)** learn initialization parameters enabling rapid few-shot adaptation to new potentials (SIGKDD). The meta-objective

optimizes for performance after few gradient steps:

$$\min_{\theta} \sum_{\mathcal{T}_i} \mathcal{L}_{\mathcal{T}_i}(f_{\theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})})$$

For GLIM, meta-learning could enable **rapid incorporation of new potentials** with minimal evaluation data, though meta-training cost is substantial.

5.3 Transfer Learning Architectures

Pre-training on abundant potential data with fine-tuning on scarce families follows standard transfer learning. The shared encoder is trained on all potentials with available data, then **frozen or fine-tuned with reduced learning rate** for data-scarce potentials. Empirical results on GLIM data show **30-50% reduction in training examples needed** when pre-training on EAM potentials (most extensive coverage) before fine-tuning on reactive or ML potentials.

Domain adversarial training encourages **family-invariant representations** by adding a discriminator that predicts potential family from embeddings, with the encoder trained to **fool this discriminator** ([Wiley Online Library](#)) . The adversarial objective:

$$\min_G \max_D [\mathcal{L}_{\text{task}}(G) - \lambda \mathcal{L}_{\text{adv}}(G, D)]$$

produces embeddings predictive of error magnitude but **uninformative about potential identity**, improving generalization to new families. For GLIM, adversarial training shows particular benefit for **empirical to ML potential transfer**, where error patterns differ substantially but structural determinants of quality are shared.

Few-shot learning for newly introduced potentials addresses practical deployment where new potentials are added with limited evaluation. **Prototypical networks** represent each potential by mean encoded features of evaluated materials, with predictions based on **similarity in embedding space**. More sophisticated approaches learn **optimal distance metrics** for error pattern discrimination.

Continual learning without catastrophic forgetting ensures adding new potentials does not degrade existing performance. **Elastic Weight Consolidation (EWC)** penalizes changes to parameters important for previous tasks; **progressive networks** add new capacity while freezing previous parameters. For evolving GLIM potential libraries, continual learning enables **incremental updates without full retraining**.

6. Attention Mechanisms for Interpretability

6.1 Attention in Crystal Graph Neural Networks

Graph Attention Networks (GAT) extend standard convolutions with **learnable neighbor importance weights** ([PyG Documentation](#)) :

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{x}_i \parallel \mathbf{W}\mathbf{x}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{x}_i \parallel \mathbf{W}\mathbf{x}_k]))}$$

$$\mathbf{x}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{x}_j \right)$$

Multi-head attention with K independent mechanisms provides **diverse perspectives on neighbor importance**, with outputs concatenated or averaged. Different heads may specialize: one on **coordination environment**, another on **bond length distributions**, a third on **local symmetry elements**.

Global attention pooling extends attention to graph-level readout, learning which atoms most influence predictions. The **Set2Set mechanism** in MEGNet implements iterative attention: LSTM-generated queries attend to node features over multiple steps, progressively focusing on relevant information (PyPI). For error prediction, global attention identifies **which structural regions drive potential failures**.

6.2 Structural Motif Identification

Local attention weights for atomic environment importance directly identify atoms whose neighborhoods most influence error predictions. Visualization of per-atom attention scores overlaid on crystal structures highlights **chemically intuitive patterns**: surface atoms for EAM surface energy failures; atoms in unusual oxidation states for charge-dependent potential errors; undercoordinated sites for coordination-sensitive potentials.

Bond-level attention extends analysis to specific interatomic interactions. The product of connected node attention weights, or explicit edge attention mechanisms, reveals **which bond types, distances, or directions are problematic**. For **shear modulus errors**, bond-oriented attention may align with **specific crystallographic directions** where the potential fails to capture elastic anisotropy.

Hierarchical attention across multiple length scales combines: (1) **local atomic attention** for coordination environments; (2) **intermediate cluster attention** for coordination polyhedra or structural motifs; (3) **global structure attention** for long-range order. This multi-scale decomposition matches the physical organization of crystals and enables interpretation at appropriate scales for different failure modes.

6.3 Potential-Specific Failure Analysis

Attention visualization for per-potential error attribution enables systematic comparison across the 23 GLIM potentials. Aggregating attention weights across materials where a given potential shows large errors reveals **characteristic failure motifs**: EAM potentials may consistently attend to **surface atoms** or **high-charge-transfer regions**; ReaxFF may focus on **bond order variation** or **reactive sites**; ML potentials may highlight **extrapolatory structures** outside training distribution.

Correlation between attention patterns and known potential limitations validates interpretability. When attention-identified failure motifs align with **documented limitations from independent sources**—EAM's neglect of directional bonding, ReaxFF's approximate dispersion treatment, ML potential extrapolation failures—confidence in attention-based explanations increases. This validation is essential for scientific deployment where interpretability claims must be substantiated.

Automated motif extraction scales attention analysis to the full GLIM dataset. **Clustering attention-weighted subgraphs** or **frequent pattern mining** on high-attention regions discovers recurring patterns not anticipated from theory. These **data-discovered motifs** complement knowledge-based analysis and may reveal unexpected failure modes informing next-generation potential development.

6.4 Explainability Techniques

Gradient-based attribution methods provide complementary perspectives. **Integrated Gradients** accumulate gradients along paths from baseline to actual input, satisfying **completeness and sensitiv-**

ity axioms ([Atlantis Press](#)) . **SHAP** (SHapley Additive exPlanations) values provide theoretically grounded feature importance from cooperative game theory. For crystal graphs, these methods attribute predictions to **specific atoms, bonds, or structural properties**.

Perturbation-based explanations assess structural sensitivity through systematic modifications: removing atoms, perturbing positions, modifying elements. While computationally expensive, perturbation provides **ground-truth attribution** validating faster attention-based approaches.

Counterfactual generation creates minimal structural modifications achieving target error predictions. For a material predicted to have large errors, counterfactuals might identify **small changes reducing predicted errors**, providing constructive guidance for **material design or potential selection**.

7. Graph Pooling Strategies for Materials Property Prediction

7.1 Global Pooling Approaches

Method	Description	Complexity	Best For	Limitations
Mean pooling	Average node features	$O(N)$	Intensive properties, baseline	Loses distribution information
Sum pooling	Sum node features	$O(N)$	Extensive properties	Size-dependent, unstable
Set2Set	Iterative LSTM attention	$O(T \cdot N \cdot d)$	Variable-size graphs, rich aggregation	Higher computation, more parameters
Attention pooling	Learned importance weighting	$O(N \cdot d^2)$	Interpretable focus on relevant regions	Risk of overfitting to spurious patterns

Mean and sum pooling provide simple, fast baselines. Mean pooling enforces **intensive property scaling** appropriate for elastic moduli; sum pooling preserves **size information** for extensive quantities. Both treat all nodes equally, potentially diluting signals when only subsets are relevant.

Set2Set pooling employs **LSTM-based iterative attention** for order-invariant aggregation ([PyPI](#)) . Multiple processing steps enable **progressive refinement**, with empirical optimum of **3-6 steps** for materials. The attention mechanism provides **interpretability**: which atoms most influence predictions is directly extractable.

Attention-based pooling learns **parameterized importance weighting**:

$$\mathbf{h}_{\text{graph}} = \sum_i \alpha_i \mathbf{h}_i, \quad \alpha_i = \text{softmax}(\mathbf{a}^T \text{MLP}(\mathbf{h}_i))$$

This enables **task-specific focus**: different potentials may attend to different structural features. The learned weights provide **direct interpretability** valuable for GLIM's diagnostic goals.

7.2 Hierarchical Pooling Methods

DiffPool learns **differentiable cluster assignment** through soft node-to-cluster mapping ([arXiv.org](#)) . The assignment matrix $\mathbf{S} \in \mathbb{R}^{N \times K}$ produces coarsened features $\mathbf{X}' = \mathbf{S}^T \mathbf{X}$ and adjacency $\mathbf{A}' = \mathbf{S}^T \mathbf{A} \mathbf{S}$.

Multiple DiffPool layers create **multi-scale representations** capturing structure from atoms to motifs to superstructures.

MinCut pooling employs **spectral clustering** for graph coarsening, optimizing normalized cut objectives. The spectral formulation provides **theoretical guarantees** and often produces **interpretable clusters** corresponding to structural units (coordination polyhedra, molecular fragments).

Top-k pooling selects nodes by **learned importance scores**, creating sparse hierarchical representations. Hard selection creates **gradient estimation challenges** addressed by continuous relaxations; soft variants retain all nodes with learned weights.

7.3 Physics-Informed Pooling

Symmetry-aware pooling respects **crystal space group symmetry**, ensuring equivalent atoms receive identical treatment. Implementation options include: **averaging over symmetry-equivalent positions** before pooling; **constraining learned attention** to respect symmetry; or **pooling over asymmetric unit** with multiplicity weighting. For high-symmetry systems, this dramatically reduces effective graph size and enforces **physically correct behavior**.

Periodicity-preserving aggregation addresses the **infinite periodic nature** of crystals. Rather than pooling over finite supercells, physics-informed approaches: pool over **asymmetric unit with proper weighting**; employ **Fourier-based representations** respecting translational symmetry; or use **equivariant operations** that commute with translations.

Property-specific pooling for elastic constant prediction recognizes that **bulk modulus** (volumetric response) and **shear modulus** (shape distortion) depend on different structural features. **Multi-task architectures with task-specific pooling layers** can adapt: global pooling for bulk modulus; **directional pooling preserving orientation information** for shear components.

7.4 Pooling Selection for Error Prediction

For the GLIM application, **attention-based pooling** typically provides the **optimal balance** of accuracy, interpretability, and computational efficiency. Hierarchical pooling (DiffPool, MinCut) offers **maximum expressiveness** for challenging cases at **substantially increased cost**, justified for detailed analysis but not large-scale screening.

Empirical comparison on **MatBench-elastic** shows attention pooling improves **shear modulus MAE** by **8-12%** over mean pooling, with **smaller gains for bulk modulus** ([Springer](#)). For **error prediction specifically**, where failure modes may be **localized**, learned pooling advantages are **more pronounced**—**15-20% improvements** for identifying materials with large potential errors.

8. Benchmarks and Performance Evaluation

8.1 MatBench and Materials Project Benchmarks

MatBench-elastic provides **standardized evaluation** for elastic constant prediction, with **fixed train/validation/test splits** and **consistent metrics** enabling fair comparison ([PyPI](#)). The dataset comprises DFT-calculated **bulk modulus (K)**, **shear modulus (G)**, and **elastic tensor components** across diverse materials.

Architecture	Bulk Modulus MAE (GPa)	Shear Modulus MAE (GPa)	Key Innovation
CGCNN	~12-15	~12-15	First crystal GNN, distance-only features
MEGNet	~10-12	~10-12	Global state, Set2Set pooling, multi-fidelity
SchNet	~11-13	~10-12	Continuous filters, end-to-end differentiable
DimeNet	~9-11	~8-10	Explicit angle embeddings, directional message passing
ALIGNN	10.40	9.48	Line graph for bond angles, state-of-the-art for scalar moduli
MatTen	7.37	8.38	SO(3)-equivariance, full tensor prediction

Performance metrics extend beyond MAE/RMSE to include: **relative error** (percentage of reference value); **R² coefficient of determination**; **percentage within 10% of reference**; and **ranking correlation** for screening applications. For elastic constants, **relative errors of 10-15%** are often acceptable given DFT uncertainties and experimental variability.

8.2 GNN Accuracy on Elastic Properties

Bulk and shear modulus prediction has achieved remarkable accuracy, with **ALIGNN** and **MatTen** approaching or exceeding experimental precision for many materials. The **5.08×** and **2.86×** improvements over MAD baselines for Kv and Gv respectively demonstrate the value of learned representations over fixed descriptors ([PyPI](#)) .

Full elasticity tensor prediction with **equivariant models (MatTen)** addresses complete **anisotropic elastic response**. The **21 independent components for triclinic systems** (reduced by symmetry for higher systems) require **tensor-output architectures** that enforce correct transformation properties. MatTen’s **30-50% error reductions for anisotropic components** compared to non-equivariant alternatives justify the increased complexity when directional response is critical ([arXiv.org](#)) .

Generalization across crystal systems remains challenging: **cubic and hexagonal systems** (high symmetry, extensive data) achieve better accuracy than **triclinic and monoclinic** (low symmetry, limited coverage). Similarly, performance varies by **chemical system**: transition metals well-represented in training data are better predicted than **actinides, rare-earths, or exotic compositions**.

8.3 GLIM/JARVIS-FF Dataset Results

Baseline models (CGCNN, MEGNet) on GLIM error prediction achieve **25-35% mean absolute percentage error** for predicting whether potential errors exceed median values, with substantial variation across potentials. **EAM potentials** (extensive coverage, systematic errors) are **most predictable**; **newly introduced ML potentials** with sparse, variable errors are **most challenging**.

Advanced architectures (**ALIGNN**, **equivariant models**) show **consistent improvements**: **15-25% error reduction** for scalar modulus errors, **larger gains for tensor components** where directional information is critical. The **line graph construction in ALIGNN** particularly benefits **shear modulus error prediction**, where bond angles dominate anisotropic response.

Multi-task versus single-task learning on GLIM data demonstrates **clear advantages for shared representations**: **10-20% accuracy improvement** for data-scarce potentials when trained jointly with data-rich related potentials, with **minimal degradation for abundant-data potentials**. Hierarchical multi-task structures with **explicit family grouping** provide additional **5-10% gains** when family assignments are accurate.

Uncertainty calibration assessment shows: **MC dropout** achieves **~80% coverage** for 90% confidence intervals (moderate underconfidence); **deep ensembles** achieve **~88% coverage** (near-ideal); **evidential networks** require more careful tuning but enable **single-pass estimation**. Calibration varies by **potential family**: empirical potentials with systematic errors are **better calibrated** than ML potentials with complex, heterogeneous error patterns.

8.4 Comparative Analysis

Architecture ablation studies on GLIM data isolate key design choices: **angle information** (DimeNet, ALIGNN) provides **largest single improvement** for shear-related errors; **equivariant representations** (MatTen) are **essential for full tensor prediction**; **attention mechanisms** improve **interpretability with modest accuracy gains**; **hierarchical pooling** helps for **localized failure modes** but adds **substantial computation**.

Transfer learning effectiveness across potential families: **pre-training on EAM → fine-tuning on MEAM** achieves **best results** (similar formalism, extensive EAM data); **empirical → reactive** shows **moderate transfer** (different formalisms but shared metallic bonding); **empirical → ML potentials** shows **limited direct transfer** but **meta-learning enables rapid adaptation**; **ab initio-derived potentials** benefit from **all prior pre-training** due to diverse training data.

Attention interpretability validation against known limitations: **EAM attention** concentrates on **surface atoms and high-charge-transfer regions**—matches known limitations; **ReaxFF attention** focuses on **bond order variation and reactive sites**—matches expected behavior; **ML potential attention** highlights **out-of-distribution structural features**—enables **proactive uncertainty quantification**.

9. Practical Implementation Guidance

9.1 Data Preprocessing with JARVIS-Tools

Crystal structure to graph conversion workflows in JARVIS-Tools provide **automated pipelines** from CIF/POSCAR files to PyTorch Geometric Data objects:

```
from jarvis.core.atoms import Atoms
from jarvis.core.graphs import Graph
```

```

# Load crystal structure
atoms = Atoms.from_cif("material.cif")

# Construct crystal graph with specified cutoff
graph = Graph.atom_dgl_multigraph(
    atoms,
    cutoff=8.0, # Å, typical for elastic properties
    atom_features="cgcnv" # 92-dimensional elemental features
)

# Access node features, edge indices, edge distances
node_feats = graph.ndata["feat"] # [N_atoms, 92]
edge_src, edge_dst = graph.edges() # edge indices
edge_dist = graph.edata["r"] # [N_edges, 1] distances

```

Feature normalization and augmentation: node features (elemental properties) are typically **standardized to zero mean, unit variance** per feature; **edge distances** are **Gaussian-expanded** with 40 basis functions spanning 0-6 Å; **data augmentation** includes **small random strains** ($\pm 1\%$) and **thermal displacement sampling** to improve robustness.

Train/validation/test splitting for robust evaluation: **compositional holdout** ensures no shared element combinations between splits; **structural holdout** by crystal system tests generalization to novel symmetries; **potential-family holdout** evaluates transfer learning; **stratified sampling** ensures representative coverage of error magnitude distributions.

9.2 Model Implementation with PyTorch Geometric

Custom dataset class for multi-label error vectors:

```

from torch_geometric.data import Dataset, Data

class GLIMDataset(Dataset):
    def __init__(self, root, transform=None, pre_transform=None):
        super().__init__(root, transform, pre_transform)

    def process(self):
        # Load JARVIS-Tools structures and GLIM error data
        for idx, (atoms, errors) in enumerate(zip(structures, error_vectors)):
            # Construct crystal graph
            graph = jarvis_to_pyg(atoms) # nodes, edge_index, edge_attr

            # Multi-task targets: 23 potentials × 6 properties (K, G, E, C11, C12, C44)
            graph.y = torch.tensor(errors, dtype=torch.float) # [23, 6]
            graph.mask = ~torch.isnan(graph.y) # observed entries

            self.save(graph, f"data_{idx}.pt")

```

Message passing layer customization for crystal graphs with periodic boundary handling:

```

from torch_geometric.nn import MessagePassing
from torch_cluster import radius_graph

```

```

class CrystalConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr="add")
        self.lin = torch.nn.Linear(in_channels, out_channels)
        self.edge_mlp = torch.nn.Sequential(
            torch.nn.Linear(in_channels * 2 + edge_dim, out_channels),
            torch.nn.SiLU(),
            torch.nn.Linear(out_channels, out_channels)
        )

    def forward(self, x, pos, batch, cutoff=8.0):
        # Build edge index with periodic boundary conditions
        edge_index = radius_graph(pos, r=cutoff, batch=batch)

        # Compute edge attributes: Gaussian-expanded distances
        edge_vec = pos[edge_index[0]] - pos[edge_index[1]]
        edge_dist = torch.norm(edge_vec, dim=1, keepdim=True)
        edge_attr = gaussian_expansion(edge_dist) # [N_edges, 40]

        # Message passing
        return self.propagate(edge_index, x=x, edge_attr=edge_attr)

    def message(self, x_i, x_j, edge_attr):
        # Concatenate node and edge features
        z = torch.cat([x_i, x_j, edge_attr], dim=1)
        return self.edge_mlp(z)

```

Multi-task output head with uncertainty:

```

class MultiTaskHead(torch.nn.Module):
    def __init__(self, hidden_dim, num_potentials=23, num_properties=6):
        super().__init__()
        # Shared feature projection
        self.shared = torch.nn.Sequential(
            torch.nn.Linear(hidden_dim, hidden_dim),
            torch.nn.SiLU(),
        )

        # Potential-specific mean and log-variance heads
        self.mean_heads = torch.nn.ModuleList([
            torch.nn.Linear(hidden_dim, num_properties)
            for _ in range(num_potentials)
        ])
        self.logvar_heads = torch.nn.ModuleList([
            torch.nn.Linear(hidden_dim, num_properties)
            for _ in range(num_potentials)
        ])

    def forward(self, h_graph):
        h = self.shared(h_graph) # [B, hidden_dim]

```

```

means, logvars = [], []
for p in range(self.num_potentials):
    means.append(self.mean_heads[p](h))      # [B, num_properties]
    logvars.append(self.logvar_heads[p](h))  # [B, num_properties]

return torch.stack(means, dim=1), torch.stack(logvars, dim=1)
# [B, num_potentials, num_properties] each

```

9.3 Code Structure and Pseudocode

Complete training loop with uncertainty-aware, masked multi-task loss:

```

def uncertainty_masked_loss(pred_mean, pred_logvar, target, mask):
    """
    Negative log-likelihood with masking for missing data.

    Args:
        pred_mean: [B, num_potentials, num_properties]
        pred_logvar: [B, num_potentials, num_properties]
        target: [B, num_potentials, num_properties] with NaN for missing
        mask: [B, num_potentials, num_properties] boolean, True if observed
    """
    # Compute precision from predicted log-variance
    precision = torch.exp(-pred_logvar)

    # Gaussian negative log-likelihood
    nll = 0.5 * (pred_logvar + precision * (target - pred_mean)**2)

    # Apply mask and normalize by number of observed entries
    masked_nll = (nll * mask).sum() / mask.sum()

    return masked_nll

# Training loop
for epoch in range(num_epochs):
    for batch in train_loader:
        optimizer.zero_grad()

        # Forward pass
        pred_mean, pred_logvar = model(batch)

        # Compute masked loss
        loss = uncertainty_masked_loss(
            pred_mean, pred_logvar,
            batch.y, batch.mask
        )

        # Backward and optimize
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

```



```

optimizer.step()

# Validation with MC dropout for uncertainty
model.train() # Keep dropout active
with torch.no_grad():
    mc_predictions = [model(val_batch) for _ in range(50)]
    # Compute epistemic variance across MC samples

```

Inference with uncertainty quantification:

```

def predict_with_uncertainty(model, data, num_samples=50):
    """
    Monte Carlo dropout for epistemic + aleatoric uncertainty.
    """
    model.train() # Enable dropout

    means, logvars = [], []
    for _ in range(num_samples):
        with torch.no_grad():
            m, lv = model(data)
            means.append(m)
            logvars.append(lv)

    # Stack samples: [num_samples, B, num_potentials, num_properties]
    means = torch.stack(means)
    logvars = torch.stack(logvars)

    # Epistemic uncertainty: variance of mean predictions
    epistemic_var = means.var(dim=0)
    mean_pred = means.mean(dim=0)

    # Aleatoric uncertainty: mean of predicted variances
    aleatoric_var = logvars.exp().mean(dim=0)

    # Total uncertainty
    total_var = epistemic_var + aleatoric_var

    return {
        'mean': mean_pred,
        'epistemic_std': epistemic_var.sqrt(),
        'aleatoric_std': aleatoric_var.sqrt(),
        'total_std': total_var.sqrt()
    }

```

9.4 Hyperparameter Optimization

Hyperparameter	Search Range	Recommended	Impact
Hidden dimension	[64, 128, 256, 512]	256	Representation capacity vs. memory

Table 8 – continued

Hyperparameter	Search Range	Recommended	Impact
Number of layers	[2, 3, 4, 6]	4	Receptive field vs. over-smoothing
Attention heads	[4, 8, 16]	8	Diversity of structural detection
Learning rate	[1e-4, 5e-4, 1e-3]	5e-4 with cosine decay	Convergence speed and stability
Batch size	[16, 32, 64, 128]	32-64	Gradient noise vs. memory
Dropout rate	[0.0, 0.1, 0.2, 0.3]	0.1 conv, 0.3 heads	Regularization strength
Weight decay	[0, 1e-6, 1e-5, 1e-4]	1e-5	Parameter magnitude control

Learning rate schedules: cosine annealing with **10-epoch warmup** from 0 to peak, then decay to minimum over 300 epochs. **AdamW optimizer** with decoupled weight decay typically outperforms standard Adam.

Architecture search: Use **Optuna** or **Ray Tune** with **multi-objective optimization** (accuracy vs. inference time). **Population-based training** enables efficient exploration of training dynamics hyperparameters.

Regularization: **Early stopping** with **20-epoch patience** on validation loss; **gradient clipping** at **norm 1.0** for stability; **label smoothing** (0.1) for calibration improvement.

9.5 Deployment and Production

Model serialization and versioning: **TorchScript** for production inference without Python dependencies; **ONNX** for cross-platform deployment; **MLflow** or **Weights & Biases** for experiment tracking and model registry.

Batch inference optimization: **Dynamic batching** grouping similar-size graphs; **mixed precision (FP16)** for 2× throughput with minimal accuracy impact; **GPU memory pooling** to reduce allocation overhead; **asynchronous data loading** with prefetching.

Integration with GLIM platform APIs:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class StructureRequest(BaseModel):
    cif_string: str
    potentials: List[str] # subset of 23 to evaluate
```

```

@app.post("/predict_errors")
async def predict_errors(request: StructureRequest):
    # Parse CIF and construct graph
    atoms = Atoms.from_str(request.cif_string, fmt="cif")
    graph = jarvis_to_pyg(atoms)

    # Run inference with uncertainty
    with torch.no_grad():
        result = predict_with_uncertainty(model, graph)

    # Format response with per-potential predictions
    response = {
        p: {
            "predicted_errors": result["mean"][i].tolist(),
            "uncertainty": result["total_std"][i].tolist(),
            "confidence": "high" if result["total_std"][i].mean() < threshold else "low"
        }
        for i, p in enumerate(request.potentials)
    }

    return response

```

10. Future Directions and Challenges

10.1 Scaling to Larger Datasets and More Potentials

The GLIM platform's current **12,000 materials and 23 potentials** represents a substantial but ultimately limited snapshot of chemical space. Scaling to **100,000+ materials** and **50+ potentials** introduces challenges in: **memory-efficient graph batching** for variable-size crystals; **distributed training** across multiple GPUs/nodes; **active learning** to prioritize most informative evaluations; and **model compression** for deployment with constrained resources. Emerging architectures like **Graphormer** and **TokenGT** that treat graphs as sequences may enable transformer-scale training on materials data.

10.2 Incorporating Active Learning for Efficient Data Collection

Given the **computational cost of DFT reference calculations** and **force field evaluations**, active learning can dramatically improve data efficiency. **Uncertainty-based acquisition**—prioritizing materials with highest epistemic uncertainty—focuses computation where model knowledge is weakest. **Diversity-based acquisition** ensures coverage of underrepresented chemical spaces. **Multi-task active learning** must balance uncertainty across 23 potentials, potentially using **multi-objective optimization** or **Pareto frontier** approaches. The goal is to **minimize total evaluation cost while maximizing model improvement**.

10.3 Physics-Constrained GNNs for Improved Generalization

Hard physical constraints—energy conservation, force consistency, thermodynamic stability—can improve generalization and reliability. **Hamiltonian neural networks** learn energy functions whose gradients automatically satisfy conservation laws. **Neural ordinary differential equations** enable continuous-time dynamics with guaranteed physical properties. **Symmetry-constrained architec-**

tures enforce exact invariance or equivariance to relevant groups ($SO(3)$, translation, permutation). For elastic constant prediction, **convexity constraints** on the elasticity tensor ensure thermodynamic stability of predictions.

10.4 Real-Time Error Prediction for Molecular Dynamics Simulations

The ultimate application is **online error estimation during simulations**, flagging when potential reliability degrades due to: **temperature-induced disorder** moving configurations outside training distribution; **stress-induced phase transformations** creating novel local environments; or **chemical reactions** forming bonds not in potential training data. This requires: **extremely fast inference** (milliseconds per configuration); **uncertainty calibration under distribution shift**; and **graceful degradation** with fallback to expensive but reliable methods when uncertainty exceeds thresholds. Integration with **LAMMPS**, **OpenMM**, or **ASE** simulation engines would enable widespread adoption.